



HEARTS ACADEMY · UNIVERSITY OUTREACH

Life at BISTEC Global

Who we are, how we work — and how you can start your career with us.

Hearts empowering business with technology.

bistecglobal.com

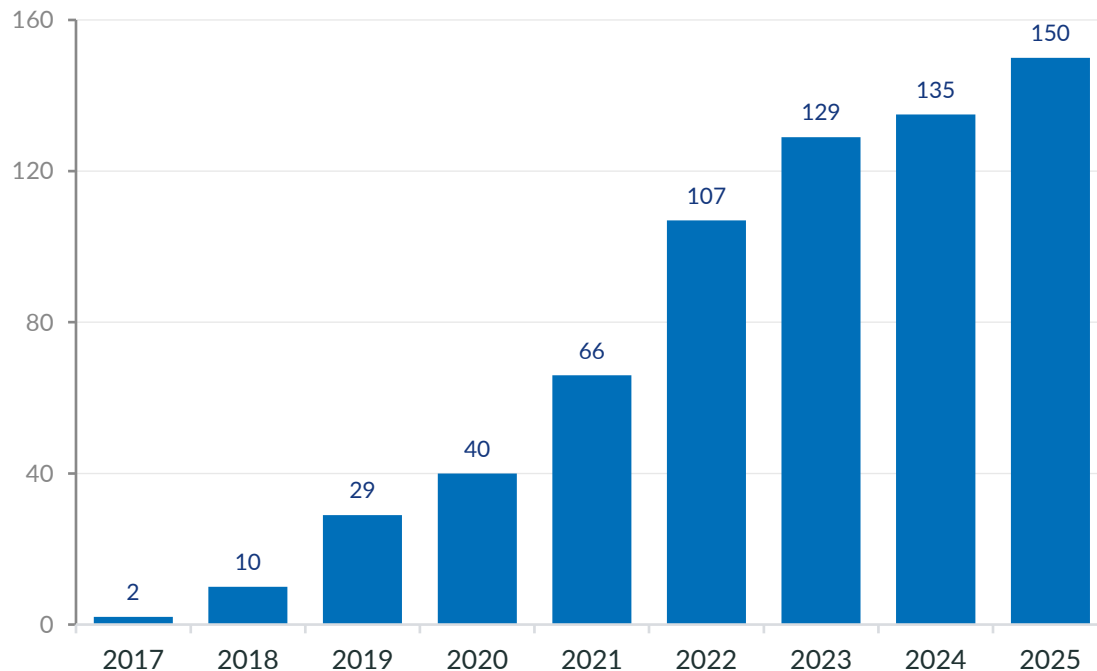


Company history

From two Hearts to one hundred and fifty – in eight years.



150 Hearts and counting



150

Hearts today

8

years of growth

2

countries — Australia
and Sri Lanka

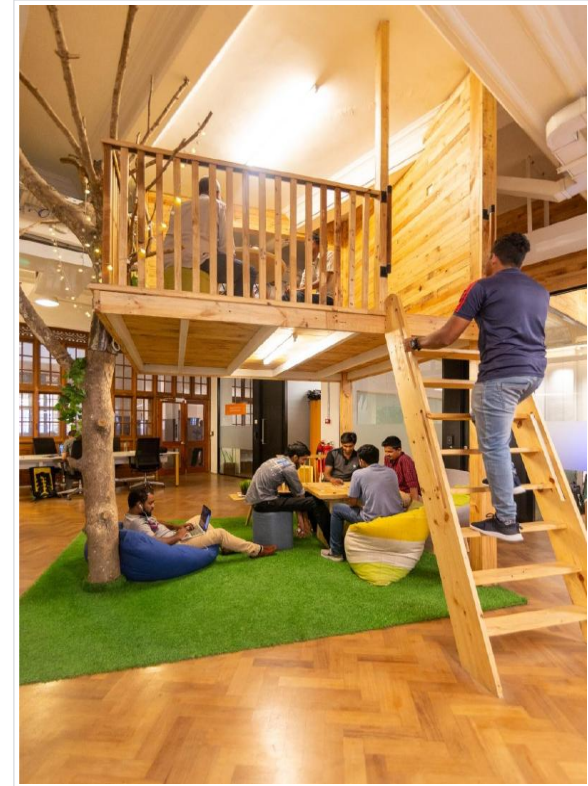
9

service brands under
one family

Where we work — BISTEC @ Hatch

- A co-working campus in the heart of Colombo 01
- Open lounges, a treehouse, quiet corners and event spaces
- Meeting rooms and studios you can book any day
- Secure, ISO 27001-aligned access and network

Come as you are. Build something real.



BISTEC Club



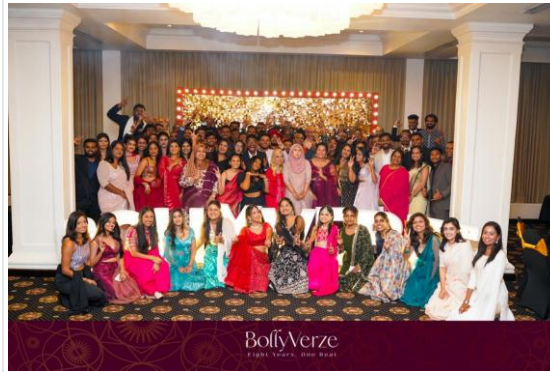
Run by Hearts, for Hearts — a committee elected fresh every year

Sport, outings, festivals and the celebrations that close out our year

Every Heart contributes a small amount each month — and the company adds one and a half times more on top.

Fun and relaxation are not an afterthought here — they have a budget and a committee.

Events @ BISTEC



Avurudu • Aloha Fiesta • award
nights • trips • expert sessions

Eight years, one beat.

BISTEC Houses

Drafted on day one. Competing all year.



HOUSE STARK

One Spirit • One Team • One Win



HOUSE LANNISTER

Teams Build Dreams



HOUSE TARGARYEN

Fearless Fighters

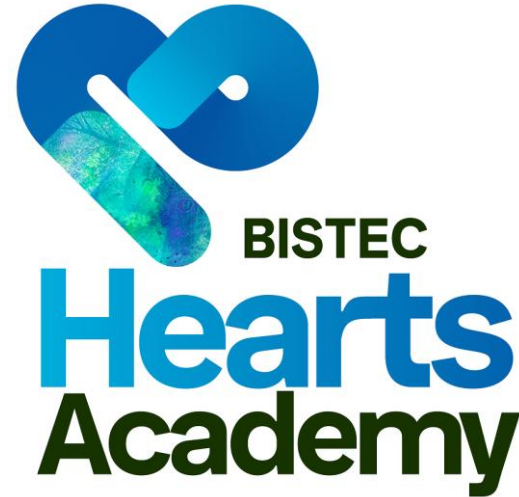


LEARNING & DEVELOPMENT AT BISTEC

What is Hearts Academy?

Where Hearts grow into engineers, speakers and leaders — every skill we need next, learnt together.

Hearts, not heads.



growing our people

Inside BISTEC – learn, build, lead



CodeCrunch Hackathons

Ignite innovation through collaboration and code.



Tech Talks

Insights, trends and tech that inspire.



Role-Based Trainings

Skill up for real-world roles and challenges.



Podcasts

Conversations that inform and inspire.



Communication Trainings

Stronger communication, stronger teams.



Public Speaking

Build confidence. Lead with impact.

Code Crunch

Our in-house coding challenge — teams, a live scoreboard, and champions crowned by Hearts Academy.



BISTEC Global AI Podcast ▶ Play all

At the intersection of human ingenuity and computational breakthrough lies the rapidly evolving world of artificial intelligence. We delve into this transformative landscape, exploring how machin...



Why devs love ❤️ Windsurf? ...
Bistec Global Services
109 views • 7 months ago



MCP | Bistec Global AI Podcast | EP 10
Bistec Global Services
170 views • 1 year ago



AI Agents | Bistec Global AI Podcast | EP 09
Bistec Global Services
77 views • 1 year ago



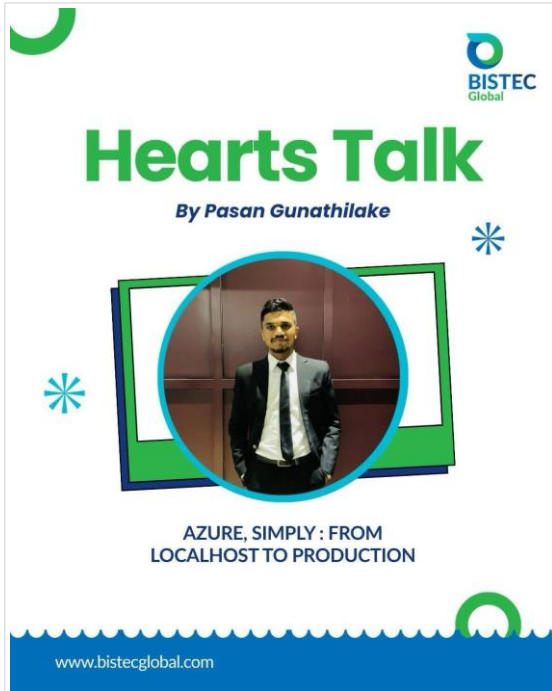
Are you sure it is real? Deepfake | Hearts Tech Show | ...
Bistec Global Services
276 views • 1 year ago



Nvidia Became the Biggest Company in the World | ...
Bistec Global Services
267 views • 1 year ago

Find it on YouTube — search “Bistec Global Services”

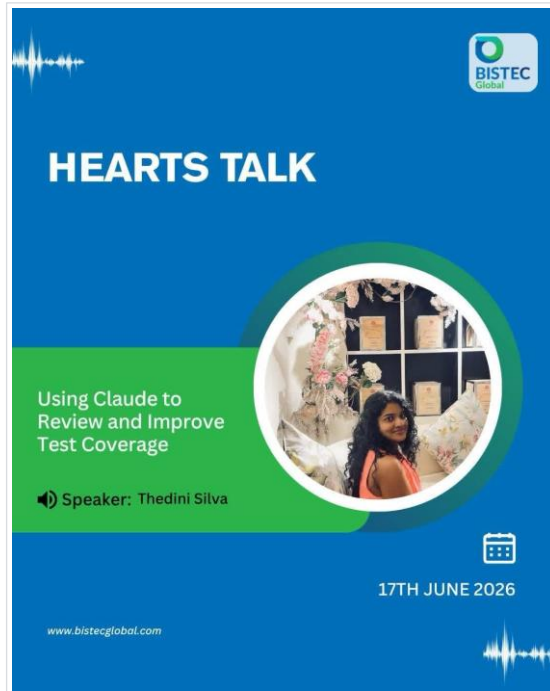
Windsurf · Model Context Protocol · AI Agents · Deepfakes · Nvidia · Hearts Tech Show



Hearts Talk
By Pasan Gunathilake

**AZURE, SIMPLY : FROM
LOCALHOST TO PRODUCTION**

www.bistecglobal.com



HEARTS TALK

Using Claude to Review and Improve Test Coverage

Speaker: Thedini Silva

17TH JUNE 2026

www.bistecglobal.com

HEARTS TALK

- Taught by our own engineers — never a vendor pitch
- Open to everyone; seniority is irrelevant
- Whatever is landing now: Azure, Claude, test coverage, AI agents
- Recorded, so nobody misses out

Your turn comes sooner than you think.

Toastmasters at BISTEC



Our own club – where engineers learn to stand up, speak clearly and lead a room.

Outside BISTEC — with campuses like yours

WE DO IT OUTSIDE



Industry Readiness Program

Preparing undergraduates for industry success.



Mentorship for Undergraduates

Guidance today, leaders tomorrow.



Industry Visits for Students

Real-world exposure. Real-world impact.



Technical Sessions on Campus

Deep dives into technology with industry experts.

Connecting universities · Creating future-ready talent

Learn. Connect. Grow. Impact. — together, we build futures.

Mentorship & Industry Visits

Undergraduates in our office — sitting with the team, seeing how the work actually gets done.



Career Fairs & Sponsorships



Follow Us!



FOLLOW IT. COMMENT IT. WIN IT.

Your BISTEC experience could win you a gift pack

1



Follow us

Follow BISTEC Global on Instagram and look out for our campaign post.

2



Comment & tag

Comment about your experience on our post and tag your friends.

3



Stand to win

Every comment enters you for a chance to win an exclusive gift.

HANDS-ON WORKSHOP · APIIT · 17 SEPTEMBER

AI for the Future

Learn. Explore. Innovate. — three engineers on what actually changed, and how to build with it.

Gayashan Wagachchige

Associate Tech Lead

Ishara Ranasinghe

Associate Software Engineer

Akansha Fernando

Associate Software Engineer



Before Any of This, We Had Stack Overflow

Ten years of building software — most of it in an era where getting unstuck meant one thing: search, try, learn.



SEARCH

Twelve tabs open. Three answers, all written in 2011.



TRY

Adapt the one that almost fits.
Watch it break. Adapt again.



LEARN

By the time it worked, you understood the problem properly.



That loop was slow. Slow is exactly what made it teach you something.

Not nostalgia, and not a complaint — just the baseline. Everything after this slide is measured against it.

It Didn't Disappear. It Compressed.

THEN

- Hours hunting for a starting point
- Read five answers, trust none
- Understanding arrived for free — as a side effect of the struggle

NOW

- Seconds to a starting point
- One answer, shaped to your problem
- Understanding has to be rebuilt on purpose — nothing hands it to you

The tool changed. The responsibility didn't.

Prompt-and-get is faster, but it skips the part where you struggled — so you have to go back for the understanding deliberately.

One Year Later

“A year ago I was told not to rely on AI. This year I watched it read a database schema it had never seen, write its own SQL, and answer a question nobody had anticipated.”



From answering to doing

Models stopped just replying and started running multi-step work with real tools.



From novelty to normal

Agentic coding tools went from a demo to part of an ordinary working day.



From consumer to builder

The interesting skill stopped being prompting and became wiring.

The advice didn't age well – and neither did my plan to just work harder than it.

Nobody is an expert yet.

Be curious. Do your research. **Start messing with it.**

THE MAIN SESSION • AKANSHA

Become an AI Engineer

Skip the Hype. Learn the Loop. Build the Agent.

Stop feeling behind on every new framework. Learn the one pattern that never changes — and leave able to build your first agent.



Who Here Feels Behind?

LangChain

LangGraph

CrewAI

AutoGen

MCP

n8n

Semantic Kernel

OpenAI Agents SDK

Claude Agent SDK

LlamaIndex

Haystack

Pydantic AI

AND NOW THE JOB DESCRIPTIONS ARE MOVING TOO

Context engineering

Loop engineering

Graph engineering

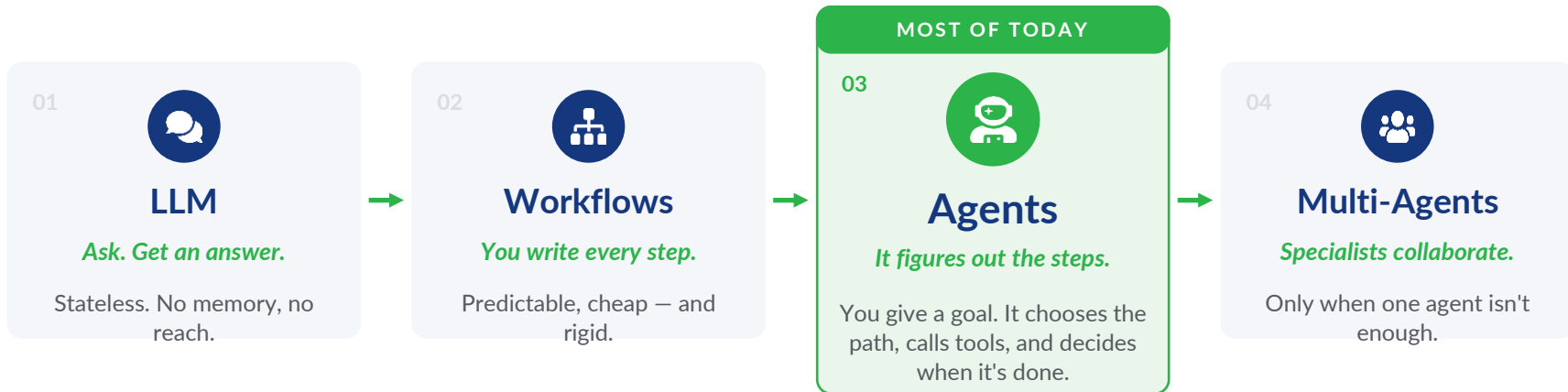
Every one of these arrived in the last 24 months. Three of them didn't exist as job titles last year.

You are not failing to keep up. You are being sold urgency.

Nobody in this room is caught up. Including me.

Four Architectures, One Session

Everything built with language models sits in one of these four boxes. Today we live in the third.



HOW WE'LL SPEND THE NEXT 40 MINUTES



The first two are context. The third is the one you'll leave able to build.

The Agent Anatomy



LLM

= *The Brain*

Thinks and reasons. Brilliant, and completely unable to do anything on its own.

+



Tools

= *Hands & Eyes*

Query a database, call an API, read a file. The ability to actually act.

=



Agent

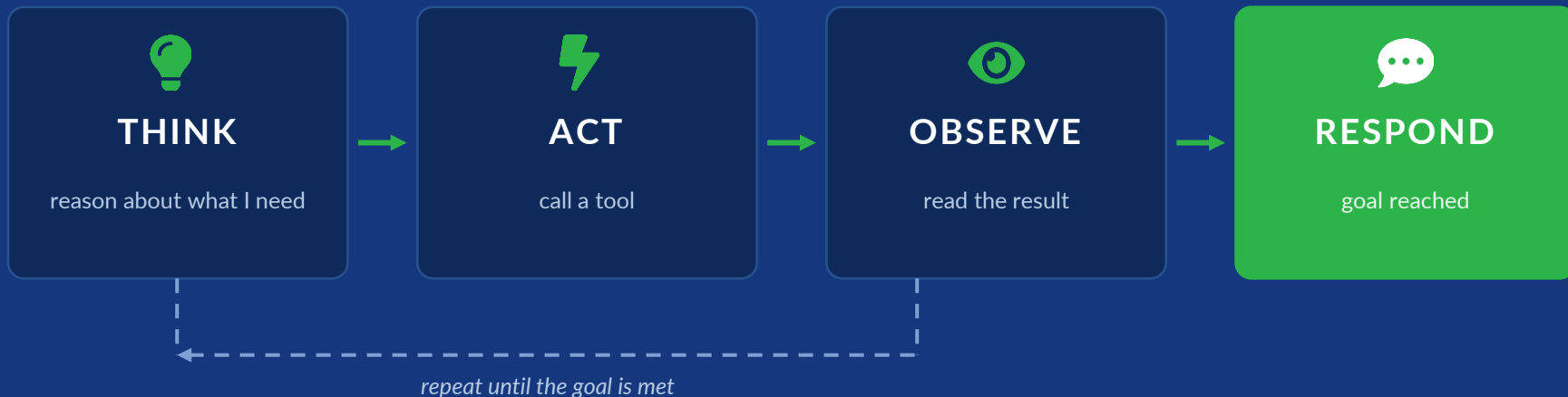
= *The Human*

Uses the brain to think and the body to act — pointed at a goal.

An agent is an LLM that can act on the world — not just talk about it.

The ReAct Loop — Reason + Act

Every agent framework you have ever heard of is this loop with different function names.



ReAct = Reason + Act. The model reasons about what to do, acts using a tool, reads what came back, and reasons again.

Search → try → learn is this same loop, run by a human. Learn it once, re-learn nothing.

Tools: What They Are, and How to Write Them

A tool is a normal function plus a description telling the model when to call it — like these three.



list_tables

What tables exist?



get_schema

What columns and types?



run_query

Execute SQL, return rows.

1. The description IS the instruction

- ✗ "Gets data"
- ✓ "Get current weather for a city.
Use when the user asks about travel."

Say what it does, what it needs, and when to reach for it. Twelve careful words can fix a 30%-wrong agent.

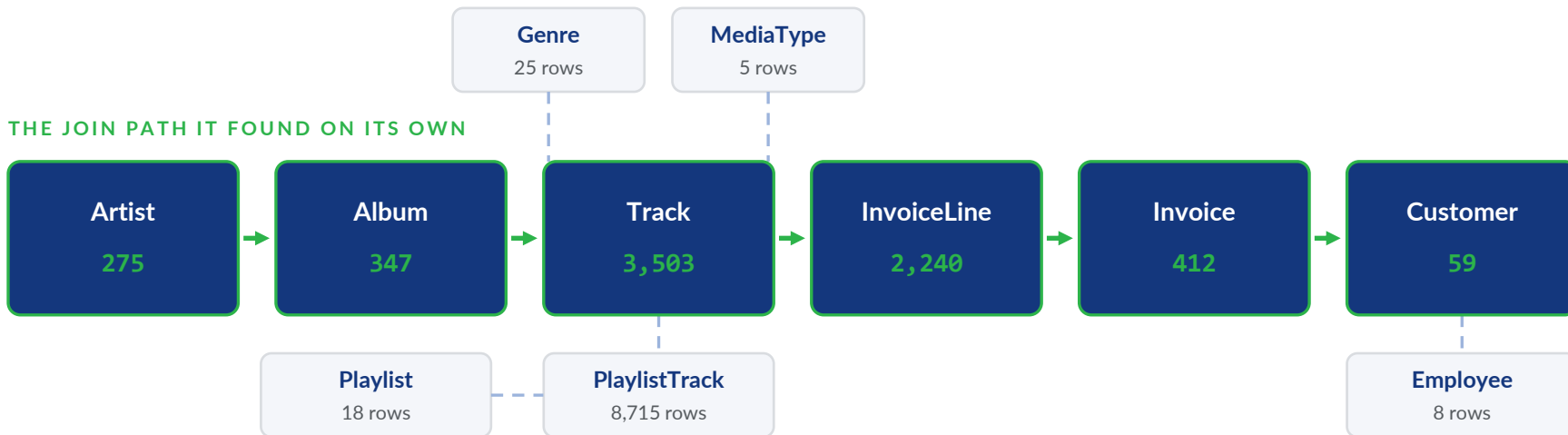
2. Why fetch, instead of sending everything

- Without tools: paste the whole database into the prompt. Huge cost, stale data, and it won't fit anyway.
- With tools: one row, fetched on demand. Cheap, live, and it scales past any context window.
- It chooses by reading the descriptions — which is why point 1 matters.

"Always inspect the schema before writing any query." — one line in my prompt. Watch it obey.

Chinook – a Music Store in 11 Tables

A standard open sample database – invoices, tracks, artists, customers. Nothing in it was built for this talk.



“Which artists have big catalogues but the lowest sales per track?” → four joins, two aggregates, one calculated metric.

I never told it these tables were related. It read the schema and worked it out.

LIVE DEMO · THE SQL AGENT

Demo Time

Same database. Same model. Now it has tools.



LangChain

Builds the agent and holds the loop



LangSmith

Shows every think / act / observe step



UI

The front end you're looking at



Claude

The reasoning, via the API

Asking: "Which artists have the largest catalogues but the lowest sales per track?"

Watch it check the schema first. Nobody told it to, except one line in the prompt.

When NOT to Use an Agent

Agents are expensive, non-deterministic, and fail creatively. Reach for one last, not first.



You already know the steps

→ **Use a workflow**

Deterministic, cheap, debuggable — the n8n canvas you saw earlier.



The action is high-stakes

→ **Put a human in the loop**

Payments, deletions, anything sent to a customer. Approve before executing.



It's a simple question

→ **Just make a plain call**

Don't pay for a reasoning loop you don't need.

Workflow first. Agent only when the steps are unknown.

Build Your First Agent This Week

```
first_agent.py

from langchain_anthropic import ChatAnthropic
from langgraph.prebuilt import create_react_agent
from langchain_core.tools import tool

model = ChatAnthropic(model="claude-sonnet-4-6")

@tool
def get_weather(city: str) -> str:
    """Get current weather for a city. Use when
    the user asks about travel or clothing."""
    return lookup(city)

agent = create_react_agent(model, [get_weather])
agent.invoke({"messages": [("user", "Wear what in Kandy?")]})
```



Starter repo

- The raw loop, hand-written — no framework
- The same agent in a few lines of LangChain
- Try out; why wait?

Build the raw loop once — so you know what the framework is hiding from you.

Build It, Post It – We'll Amplify It

An open offer, not a formality. Get your first agent running – even straight from the starter repo – and we will put our audience behind it.



01

Build it

Clone the starter repo and get it running. Change one thing – a new tool, a different database, your own question.



02

Post it on LinkedIn

Write what you built and what broke on the way. Tag BISTEC Global, or any of the three of us.



03

We repost it

We share it with our network, and we comment properly – not just a like.

Why we're offering this: a post with a working demo and an honest write-up is worth more in an interview than a line on a CV – and most students never write one.

Your first agent is also your first portfolio piece. Don't let it die in a folder.

Scan This for the Q & A



Ask anything — about agents, the demo, the code, or the career questions we didn't get to.

BUILDING REAL THINGS • ISHARA

From One Agent to a Way of Working

You've seen one agent run. Here's how we wire them into how software actually gets built.



Tools, Wiring, and the Order You Do Things In

1 Tools

One thing the AI can do. Read a file.
Search Jira. Run a test.

2 MCP

A standard way to plug a tool in.
Build it once, and any model or client
can use it.

*It doesn't make the AI smarter. It stops you
rewiring.*

3 Workflow

The order you do things in, and what
you're not allowed to skip.

*Having the tools isn't the same as knowing
the order.*

The rest of this is about that third box.

Four Ways People Work With Agents

1

Just prompt it

No structure. Ask, get code, move on.

Fast. Nobody can say why the code looks like that.

2

Rules files

A CLAUDE.md or similar. The agent learns your conventions.

Better output. Still no record of any single change.

3

Plan, then build

The agent proposes, you approve, then it writes code.

You get a say. The plan is thrown away afterwards.

4

Spec-driven

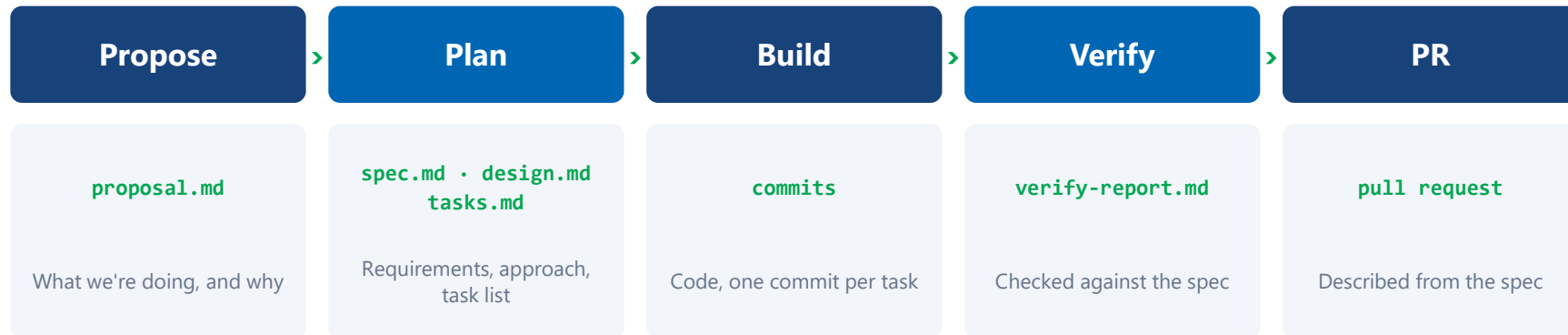
Every change leaves files behind — requirements, design, tasks, verification.

Reviewable, and the code gets checked against it.

The whole industry is somewhere on this line right now. Public examples of the fourth kind include GitHub's Spec Kit and AWS Kiro — and BISTEC has built its own.

SpecClaw — Five Stages, Five Files

Bistec Global framework. Open source, MIT, on GitHub under [bistec-oss/specclaw](#).



Each file is the input to the next stage. The spec isn't filed and forgotten — it's what the design is built from, and what the code gets checked against at the end.

Next: one real change, and the files it left behind.

Three Commands, Inside Claude Code

```
/plugin marketplace add chan4lk/specclaw
```

```
/plugin install specclaw@chan4lk
```

```
/specclaw:init
```

Before you start

- Claude Code 2.1 or later
- On Windows, jq must be on your PATH
- Run init inside the repo, not above it

init creates the **.specclaw** folder you've been looking at for the last ten minutes. It's open source — MIT, on GitHub under [bistec-oss/specclaw](#).

Four Things, Whatever You End Up Using

1 Write the criteria before you build

Otherwise "done" is just an opinion.

2 Keep the artefacts in the repo

Next to the code. Not in a tool, not in a chat history.

3 Order work by what depends on what

A dependency graph, not a checklist.

4 When it fails, write down why

Somewhere the next change can see it.

None of that requires SpecClaw. SpecClaw is what it looks like when someone takes all four seriously at once.

Meanwhile, On The Internet



Overused. Still basically right.

Which raises the question everyone in this room actually came to ask.

CAREER GUIDANCE • LED BY GAYASHAN

So Where Does This Leave You?

Ten minutes of honest answers, from one year in and ten years in.



The Engineering Mindset

The tools have changed twice since I started. The way you have to think about a problem hasn't changed once.



Break it down

Any problem you can't explain in parts is a problem you don't understand yet. This is the whole job, and no model does it for you.



Distrust the output

Yours, your teammate's, the model's. "It ran" is not "it works." Ask what would have to be true for this to be wrong.



Own the outcome

Not the code — the outcome. When it breaks at 2am, nobody accepts "the AI wrote that part."

AI made writing code cheap. It made thinking clearly more valuable, not less.

Breadth First, Depth Later

Pick a specialism too early and you optimise for a shape of job that may not exist by the time you graduate. See the whole system first.



Full Stack

How a request actually travels — browser to database and back. The mental map everything else hangs off.



QA

How software really fails. Learn this and you stop trusting output — including the model's.



Data Engineering

Where the data lives, how it's shaped, why it's dirty. Every AI system is a data problem wearing a costume.



DevOps

How code reaches a real user and stays up. The gap between “it works locally” and “it works.”

Go deep once you know which part of the system you actually enjoy — not because a job ad told you to.

You cannot specialise in something you've never seen the shape of.

Fundamentals Don't Expire

Everything you learn has a shelf life. Choose what you spend your evenings on accordingly.



Expires in 18 months

- This week's framework
- A specific model's quirks
- Prompt tricks that happen to work
- Whichever IDE is fashionable



Still true in ten years

- Data structures and complexity
- How the network actually works
- How databases store and index
- How to debug something you didn't write
- How to write plainly for humans

The right-hand column is what lets you pick up the left-hand column in a weekend.

Frameworks are rented. Fundamentals are owned.

Spec-Driven Development — and the Loop

Two habits, one idea underneath: decide before you generate.



Write the spec first

- Say what should be true when you're done — before a line is written.
- The spec is what you review; the code becomes the easy part.
- It's how BMAD, Spec Kit and SpecClaw all work — you saw it earlier.



Learn the loop

THINK

ACT

OBSERVE

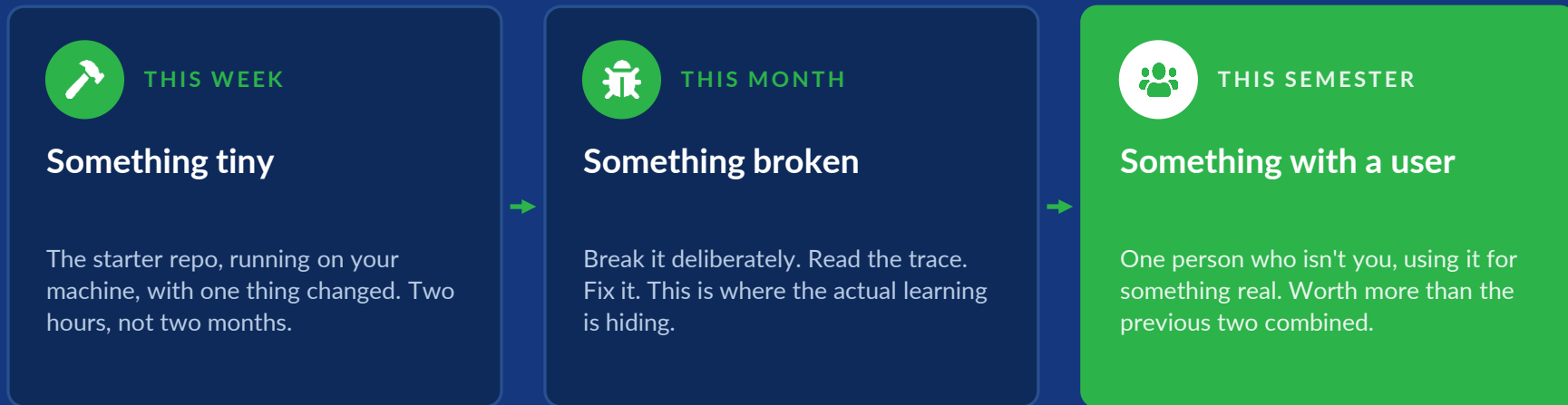
Every agent, every framework, every year. Learn it once and each new release becomes a documentation problem instead of a panic.

A spec is a plan a human approves. The loop is how the work gets done. You need both.

Anyone can generate code now. Deciding what should exist is the job.

Build. Build. Build.

Nothing on the last four slides happens by reading about it. There is no certificate for this.



Every engineer you admire got there by shipping things that embarrassed them a year later.

Start this week. Small and broken beats planned and imaginary.

Two Honest Answers



“Will AI take entry-level developer jobs?”

The work is shifting from writing every line to specifying, reviewing and being accountable for what gets written. That shift is real, not hypothetical. What's scarce right now is people who can judge whether an AI system's output is actually correct — which needs real domain understanding.

Nobody honest knows how this looks in five years. Anyone who claims certainty is guessing.



“Do I need to be good at maths?”

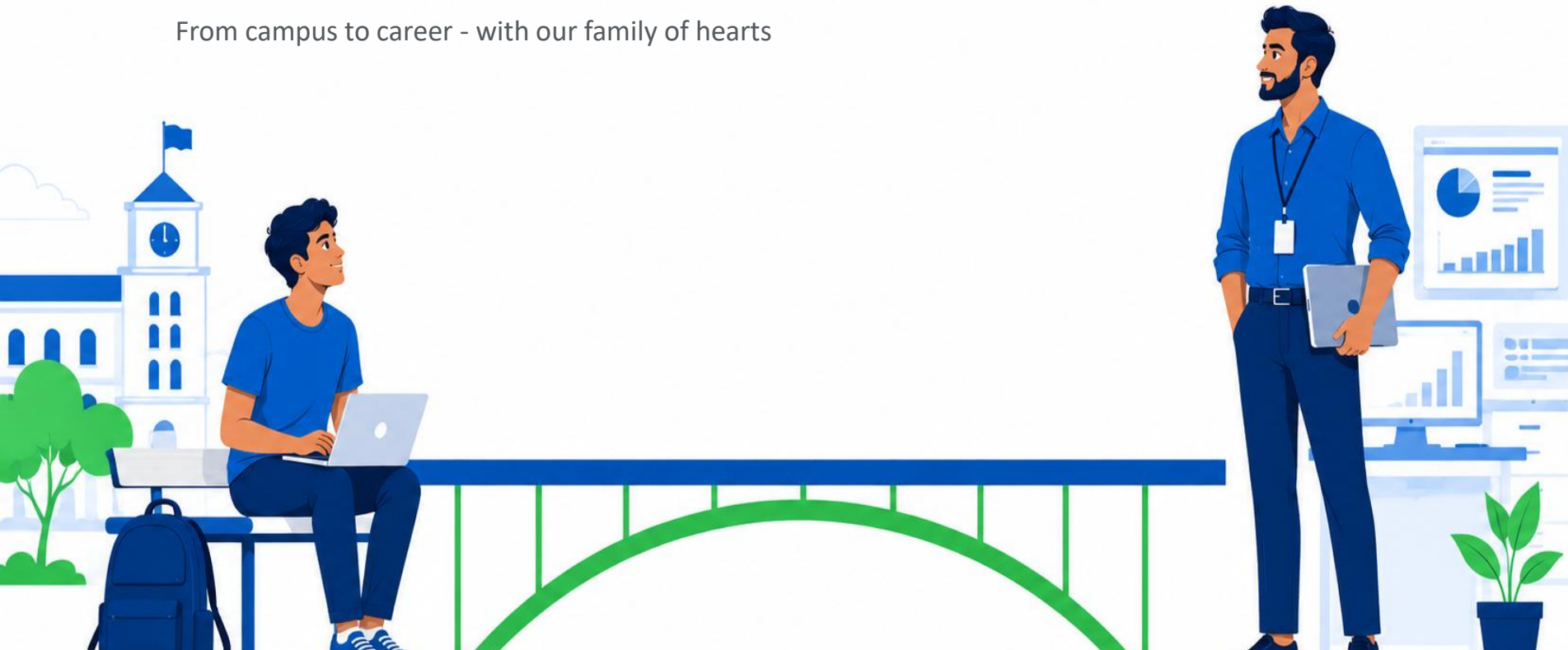
Training a model needs maths. Using one doesn't. An AI engineer writes normal software — functions, APIs, error handling — around a model somebody else trained.

You do need maths for research, fine-tuning or inference work. That's a different job title, and it's fine not to want it.

“I don't know, but here's how I'd find out” is a strong answer. Use it.

Industry Readiness Program

From campus to career - with our family of hearts



Who You'd Be Learning From

The same engineers who ship for our clients are the ones teaching in this program.

150+

Hearts in our family

8+ yrs

Custom software delivery

15+

Countries, 4 continents

3rd

Best Workplace, Sri Lanka
2024

NINE BRANDS, ONE FAMILY

Agent Accelerator

BISTEC Care

Denstro

Accounting

Social Media

Hearts empowering business with technology.



What You Can Become



01

Industry-Ready Engineer



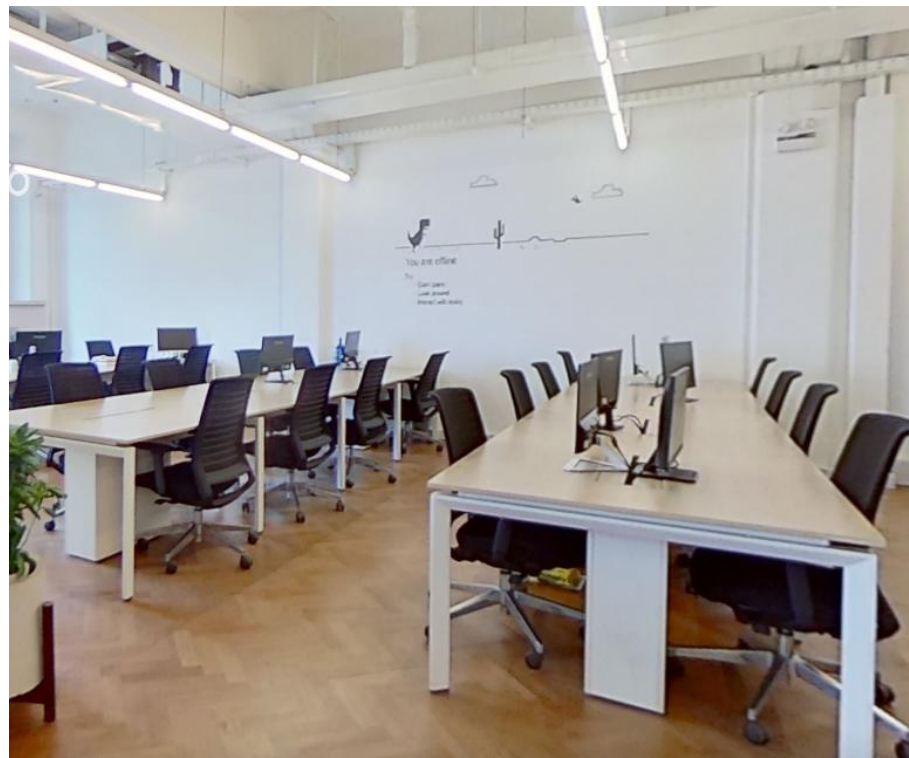
02

Placed at BISTEC or a Partner



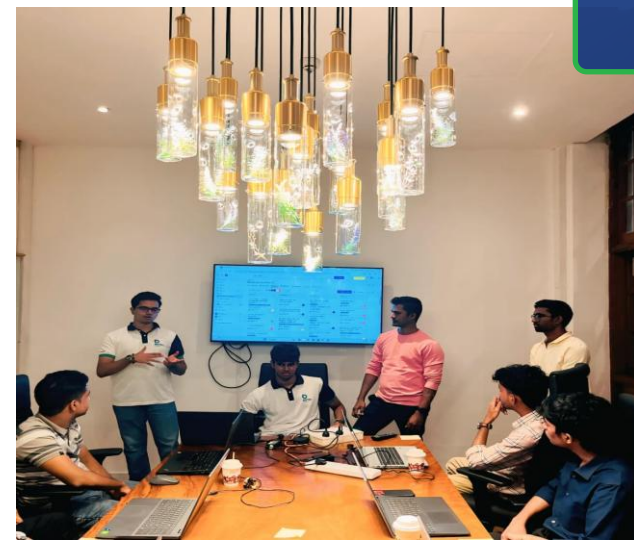
03

Skillful Entrepreneur



6 months = NAITA approved internship.

We started on May 12th 2026



Our University Partners

BISTEC Reach takes the program to campuses directly - so students meet industry before they graduate.



**INFORMATICS
INSTITUTE OF
TECHNOLOGY**



Curtin University

Colombo



**Gampaha Wickramaraachchi
University of Indigenous Medicine**



**General Sir John Kotelawala
Defence University**

For the Motherland Forever | සිය රටටමයි කඳුන්



A Senior Engineer in Your Corner



Guidance through experience.



HEARTS TALK

AZURE, SIMPLY: FROM LOCALHOST TO PRODUCTION



PASAN GUNATHILAKA

08/07/2026 05:00 PM ONWARDS MS TEAMS



HEARTS TALK

AI TO AGI: HOW CLOSE ARE WE TO HUMAN-LEVEL INTELLIGENCE?



NETHMI EDIRISOORIYA

03/08/2026 05:00 PM ONWARDS MS TEAMS



HEARTS TALK

INSIDE THE MIND OF AN AI



LAHIRU INDRAJITH

19/08/2026 05:00 PM ONWARDS MS TEAMS



Hearts Talk

Using Claude to Review and Improve Test Coverage

17th June 2026 05:00 PM Onwards



Thedini Silva



Hearts Talk

Claude in the Wild: From One Model Call to a Multi-Agent System

3rd June, 2026 05:00 PM Onwards



Akansha Fernando



Hearts Talk

Building Your First MCP Server: Giving AI Agents Real-World Tools

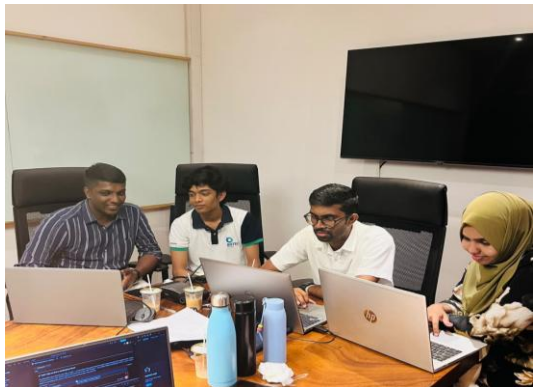
26th of May 2026 05:00 PM Onwards



Thameera Tharmakulasingam

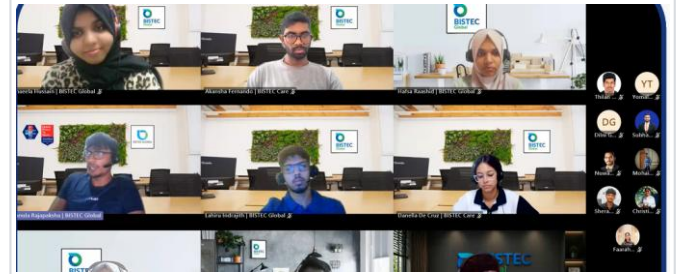


Code Crunch: Champions in the Making



Finding Their Voice

Engineers who can explain their work get trusted with bigger work.



The best code in the room loses to the person who can explain theirs.

Scan for Whatsapp

076 8030 944



Scan This for the Quiz



Ten questions from this session. Easy, medium, hard — the hard ones are worth triple.